Soprillo: Small, but Mechanized SAX

ARMAN AHMED, McGill University, Canada
MAX GROSS, McGill University, Canada
FRÉDÉRIC LAHAIE-BERTRAND, Université de Montréal, Canada

This paper presents a work-in-progress mechanization of the proof theory of the semi-axiomatic sequent calculus (SAX) in the proof assistant Beluga. SAX replaces all noninvertible inference rules with axioms, causing standard cut elimination to fail. Instead, SAX satisfies a modified cut elimination theorem permitting certain analytic cuts called snips to remain, where a snip requires the cut formula to be 'eligible', i.e. a subformula of the principal formula of an axiom. A central challenge in mechanizing SAX is therefore tracking eligibility throughout the proof theory. We address this by representing eligibility as a well-formedness predicate embedded directly within proof judgements rather than imposed through a separate context discipline, adapting a technique due to Crary for encoding substructural constraints as judgements over proof terms. This is analogous to how linearity conditions have been previously handled in mechanizations of session-typed systems. Using this technique, we present mechanized proofs of cut admissibility and the subformula property for a representative propositional fragment, and outline a roadmap for extending the mechanization to the full calculus. To our knowledge, this is the first mechanization of SAX proof theory.

ACM Reference Format:
Arman Ahmed, Max Gross, and Frédéric Lahaie-Bertrand. 2026. Soprillo: Small, but Mechanized SAX. 1, 1 (April 2026), 10 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

The Curry-Howard isomorphism shows the deep relationship between intuitionistic logic and computation. Specifically, it establishes that proofs can be interpreted as programs and proof reduction as computation. Because of this, we can see that the different formalizations of intuitionistic logic will result in different computational interpretations [5]. Howard [8], for example, established the connection between natural deduction and the simply-typed λ-calculus. Curry [3] took a different approach, associating Hilbert-style proofs to combinators. Herbelin [7] established the system LJT based on the sequent calculus with a stoup, leading to a computational interpretation featuring explicit substitutions.

Adding to this lineage, DeYoung et al. [5] presents the semi-axiomatic sequent calculus (SAX), a formalization of intuitionistic logic which arises by replacing all non-invertible rules in the sequent calculus with axioms and leaving the invertible rules unchanged. The resulting system is equivalent to G_3, but standard cut elimination fails: it is replaced by a modified form in which certain analytic cuts, called snips, are permitted to remain in normal-form proofs while preserving the subformula property.

Establishing these results is more involved than in the sequent calculus. Both the admissibility of cut and the subformula property require tracking which formula occurrences are eligible for a snip and which antecedents

Authors' addresses: Arman Ahmed, arman.ahmed@mail.mcgill.ca, McGill University, Montréal, Canada; Max Gross, max.gross@mail.mcgill.ca, McGill University, Montréal, Canada; Frédéric Lahaie-Bertrand, frederic.lahaie-bertrand@umontreal.ca, Université de Montréal, Montréal, Canada.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Association for Computing Machinery.
XXXX-XXXX/2026/4-ART $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

are irrelevant, and both proofs require generalized induction hypotheses to account for how the eligibility of formula occurrences changes through derivations. The original paper necessarily leaves out many portions of the proofs implicit, making SAX an interesting and non-trivial target for mechanization.

As such, we report on a mechanization [1] of the proof theory of Soprillo, a fragment of SAX in Beluga, a proof assistant with its foundations in contextual modal logic and the Logical Framework. Beluga's explicit handling of contexts is particularly suited to mechanizing SAX, precisely because we need to reason about variables in contexts during our proof. Further, since it is an extension of LF, we can build on the existing literature regarding mechanizing similar systems and proofs in the literature [2] [11]. This, to the best of our knowledge, is the first mechanization of SAX and its proof theory. Specifically, we present:

- A mechanization of Soprillo and its translation to and from G_3
- A mechanized proof of the admissibility of cut
- A mechanized proof of cut elimination
- A mechanized proof of the subformula property

The remainder of the paper is organized as follows: Section 2 motivates the mechanization through SAX's computational interpretation and illustrates the proof-theoretic subtleties that make it a challenging target. Section 3 presents our mechanization in detail. Section 4 discusses related work and Section 5 concludes.

2 MOTIVATION

2.1 The Semi-Axiomatic Sequent Calculus

The rules of SAX, where the noninvertible rules have been replaced by axioms and the invertible rules are inherited from G_3, are given in Figure 1.

$$\frac{}{\Gamma, A \vdash A}\ \mathsf{Id}_A \qquad \frac{\Gamma \vdash A \quad \Gamma, A \vdash C}{\Gamma \vdash C}\ \mathsf{Cut}_A$$

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \supset B}\ \supset R \qquad \frac{}{\Gamma, A, A \supset B \vdash B}\ \supset L^0$$

$$\frac{}{\Gamma, A, B \vdash A \otimes B}\ \otimes R^0 \qquad \frac{\Gamma, A \otimes B, A, B \vdash C}{\Gamma, A \otimes B \vdash C}\ \otimes L$$

$$\frac{}{\Gamma \vdash \mathbf{1}}\ \mathbf{1}R^0 \qquad \frac{\Gamma, \mathbf{1} \vdash C}{\Gamma, \mathbf{1} \vdash C}\ \mathbf{1}L$$

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \,\&\, B}\ \&R \qquad \frac{}{\Gamma, A \,\&\, B \vdash A}\ \&L_1^0 \qquad \frac{}{\Gamma, A \,\&\, B \vdash B}\ \&L_2^0$$

$$\frac{}{\Gamma, A \vdash A \vee B}\ \vee R_1^0 \qquad \frac{}{\Gamma, B \vdash A \vee B}\ \vee R_2^0 \qquad \frac{\Gamma, A \vee B, A \vdash C \quad \Gamma, A \vee B, B \vdash C}{\Gamma, A \vee B \vdash C}\ \vee L$$

Fig. 1. The Semi-Axiomatic Sequent Calculus (Sax) [5].

As discussed in the introduction, standard cut elimination fails because the axioms introduce formulas that cannot be unpacked. Consider the following proof, where the cut on B cannot be eliminated:

$$\dfrac{\dfrac{\dfrac{}{A, A \supset B \vdash B}\ \supset L^0 \quad \dfrac{}{B, B \supset C \vdash C}\ \supset L^0}{A, A \supset B, B \supset C \vdash C}\ \mathsf{Cut}_B}{A \supset B, B \supset C \vdash A \supset C}\ \supset R$$

[1] Available at https://github.com/l4haie/mech-sax.

However, the cut is analytic: B is a subformula of the conclusion sequent, and thus the proof exhibits the subformula property. SAX therefore restricts to such analytic cuts, called snips, which require that the cut formula in at least one of the premises originates from an axiom as a subformula of its principal formula. With their addition, SAX satisfies the subformula property [5].

In order to track the origins of each formula, [5] introduces a labelling system, marking formulas $y^* : A$ as eligible for a snip. We can then propagate this through the proof by marking a variable as eligible in the conclusion if it is eligible in at least one of its premises [5]. Since the application of axioms can contain antecedents that are irrelevant (antecedents that are not strictly needed for the application of said axiom) we must also track these irrelevant formulas by marking them $x^0 : B$. We mark a variable as irrelevant in the conclusion if it is irrelevant in both of its premises [5]. Note that eligibility and irrelevance are properties of formula occurrences in a derivation, not part of the syntax of sequents. The corresponding labeled proof from above, where $t^* : B$ is eligible because it is a subformula of the principal formula of (both) $\supset L^0$ axioms, is given below:

$$\dfrac{\dfrac{\dfrac{}{u^* : A,\ x : A \supset B,\ y^0 : B \supset C \vdash t^* : B}\ \supset L^0 \qquad \dfrac{}{u^0 : A,\ x^0 : A \supset B,\ t^* : B,\ y : B \supset C \vdash w^* : C}\ \supset L^0}{u^* : A,\ x : A \supset B,\ y : B \supset C \vdash w^* : C}\ \mathrm{Snip}_B}{x : A \supset B,\ y : B \supset C \vdash z : A \supset C}\ \supset\mathrm{R}$$

Figure 2 gives the labeled rules for Soprillo, the subset of SAX that we mechanize, covering identity (`id`), implication ($\supset R$, $\supset L^0$), tensor ($\otimes R^0$, $\otimes L$), and snips ($\texttt{snip}^1$, $\texttt{snip}^2$).

$$\dfrac{}{\Gamma^0,\ x : A \vdash y : A}\ \mathrm{Id}_A \qquad \dfrac{\Gamma \vdash x : A \quad \Gamma,\ x : A \vdash z : C}{\Gamma \vdash z : C}\ \mathrm{Cut}_A$$

$$\dfrac{\Gamma \vdash x^* : A \quad \Gamma,\ x : A \vdash z : C}{\Gamma \vdash z : C}\ \mathrm{Snip}^1_A \qquad \dfrac{\Gamma \vdash x : A \quad \Gamma,\ x^* : A \vdash z : C}{\Gamma \vdash z : C}\ \mathrm{Snip}^2_A$$

$$\dfrac{\Gamma,\ y : A \vdash z : B}{\Gamma \vdash x : A \supset B}\ \supset R \qquad \dfrac{}{\Gamma^0,\ y^* : A,\ x : A \supset B \vdash z^* : B}\ \supset L^0$$

$$\dfrac{}{\Gamma^0,\ y^* : A,\ z^* : B \vdash x : A \otimes B}\ \otimes R^0 \qquad \dfrac{\Gamma,\ x : A \otimes B,\ y : A,\ z : B \vdash w : C}{\Gamma,\ x : A \otimes B \vdash w : C}\ \otimes L$$

Fig. 2. Labelled SAX [5].

2.2 Computational Interpretation of SAX

Beyond its proof-theoretic results, SAX admits a natural operational interpretation as a system of concurrent processes that interact asynchronously. This interpretation has been studied across several computational models, from shared memory concurrency [5] to message-passing concurrency [], and each of these interpretations has been shown to satisfy desirable properties such as progress, preservation, confluence, and termination. Specifically, under the shared memory interpretation, variables are substituted by addresses at runtime, and thus the sequent

$$x_1 : A_1, \ldots, x_n : A_n \vdash P :: (z : C)$$

should be understood as typing a process P that reads from addresses $x_1, \ldots, x_n$ and writes to address z. The operational behaviour of processes is then determined by the rules of the proof system, for instance cut and snip

allocates a fresh memory cell, spawning a writer process and a reader process that synchronize through it, and process expressions assigned to right rules write to memory while those assigned to left rules read from it.

This low-level interpretation of SAX further gives rise, in extensions of SAX, to type-theoretically justified compiler optimizations such as in-place memory reuse []. Extensions of SAX in adjoint logic have also served as the logical foundation for concurrent language implementations []. All of these applications ultimately depend on the soundness of SAX's proof theory.

2.3 Mechanization

This proof theory is, however, not trivial to prove. The original paper establishes the key proof-theoretic results, i.e. the admissibility of cut, cut-elimination, and the subformula property, but necessarily leaves a significant portion of the proofs implicit, due to their complexity. The central difficulty, as discussed above is

3 CONTRIBUTIONS

3.1 Core Proof-Theoretic Results

We mechanize the following proof-theoretic results for a representative subset of SAX, covering identity, implication ($\supset R$, $\supset L^0$), tensor ($\otimes R^0$, $\otimes L$), and snips ($\texttt{snip}^1$, $\texttt{snip}^2$). The remaining connectives of SAX ($\vee$, $\&$, $\mathbf{1}$) follow the same patterns and are left as future work.

Theorem 1 (Translations between G_3 and SAX). *$\Gamma \vdash A$ in G_3 iff $\Gamma \vdash A$ in SAX.*

In the following theorems, a derivation is *cut-free* if it contains no uses of cut but may contain snips.

Theorem 2 (Admissibility of Cut). *If there are cut-free derivations of $\Gamma \vdash x : A$ and $\Gamma, x : A \vdash z : C$, there exists a cut-free derivation of $\Gamma \vdash z : C$.*

Theorem 3 (Cut Elimination in SAX). *If there is a derivation of $\Gamma \vdash x : A$, there is a cut-free derivation of the same sequent.*

Theorem 4 (Subformula Property). *All formulas in a cut-free derivation of $\Gamma \vdash x : A$ are subformulas of formulas in Γ or A.*

3.2 Encoding SAX in Beluga

We encode SAX in Beluga using HOAS. Hypotheses are represented as variables in a context of schema `lctxS` with entries of type `lhypS A`. Derivations of $\Gamma \vdash A$ are terms of type `[Γ ⊢ lconcS A[]]`. Binding in rules such as $\supset R$ is handled by LF function types, so weakening and substitution are inherited from the framework. The constructors of `lconcS` correspond one-for-one to the rules of SAX; adequacy holds because `lhypS` has no constructors, so its only inhabitants are context variables.

```
LF lhypS : o → type = ;
LF lconcS : o → type =
  | linitialS : {h : lhypS A} lconcS A
  | l⊃RS      : (lhypS A → lconcS B) → lconcS (A ⊃ B)
  | l⊃L0S     : lhypS A → lhypS (A ⊃ B) → lconcS B
  | lsnip1S   : {D : lconcS A} elig_c D → (lhypS A → lconcS B) → lconcS B
  | lsnip2S   : lconcS A → {E : lhypS A → lconcS B} elig_h E → lconcS B
  | l⊗R0S     : lhypS A → lhypS B → lconcS (A ⊗ B)
  | l⊗LS      : (lhypS A → lhypS B → lconcS C) → lhypS (A ⊗ B) → lconcS C
```

One notable feature of the encoding concerns the two variable annotations from the paper's labeled calculus. Variables marked as *irrelevant* (superscript 0) are those that appear as side formulas of an axiom or identity and are never used in the derivation. In Beluga this annotation is unnecessary: because LF has structural weakening, unused hypotheses can simply be ignored. A derivation term in context $[\Gamma, x : \texttt{lhypS}\ A]$ where x does not appear free can always be weakened back to $[\Gamma]$, directly corresponding to dropping an irrelevant antecedent. The irrelevance superscripts in the paper are therefore absorbed into the framework's own treatment of unused variables.

3.3 Eligibility as a Predicate on Derivation Terms

The central design challenge is representing *eligibility*. In the paper, a variable occurrence in a derivation is eligible if it was introduced by an axiom in an eligible position. The two forms of snip differ in which premise must be eligible: $\texttt{Snip}^1$ requires an eligible succedent in the first premise, while $\texttt{Snip}^2$ requires an eligible antecedent in the second.

A natural first attempt is to represent eligibility as a predicate on hypothesis variables directly, `LF elig_h : lhypS A -> type`, adding eligibility witnesses to the context whenever a hypothesis is introduced at an eligible position. This fails because LF contexts in Beluga are structural: weakening is always available, so an `elig_h h` witness could be assumed for any variable without it having been introduced by an axiom. The subformula property would then break, since an arbitrarily weakened eligibility witness carries no information about which axiom the variable came from, making it impossible to extract a subformula relationship for the cut formula's type.

Instead we define eligibility as predicates indexed directly over derivation terms:

```
LF elig_c : lconcS A → type =
  | elig_c/l⊃L0S : {x : lhypS A} {y : lhypS (A ⊃ B)} elig_c (l⊃L0S x y)
and
LF elig_h : (lhypS A → lconcS B) → type =
  | elig_h/l⊃L0S   : {y : lhypS (A ⊃ B)} elig_h (\x. l⊃L0S x y)
  | elig_h/l⊗R0S/1 : {z : lhypS B}       elig_h (\y. l⊗R0S (y : lhypS A) z)
  | elig_h/l⊗R0S/2 : {y : lhypS A}       elig_h (\z. l⊗R0S y (z : lhypS B))
  | elig_h/l⊗R0S/3 :                     elig_h (\y. l⊗R0S y y)
```

`elig_c D` holds when the closed derivation `D` ends in an axiom with an eligible succedent. `elig_h E` holds when the function `E` maps its abstracted variable to an axiom in which that variable occupies an eligible antecedent position. Because the certificates are indexed over derivation terms rather than variables, they cannot be introduced by weakening. This is what makes the subformula property go through: the `elig` evidence identifies the specific axiom the eligible formula came from, and from that a subformula witness can be read off directly.

The four constructors of `elig_h` enumerate every way a single variable can appear eligibly in an axiom in our fragment. Three are routine. The fourth, `elig_h/l⊗R0S/3`, covers the degenerate case where both arguments of $\otimes R^0$ are the same variable, i.e. a derivation $\Gamma^0, x^* : A \vdash x : A \otimes A$. This arises due to contraction in the intuitionistic setting and has no analogue in a linear system. Extending the mechanization to additional connectives requires only adding the corresponding `elig_h` constructors for each eligible position in the new axioms, along with the matching cases in the proofs below.

This approach is related to the linearity predicates of Sano, Kavanagh, and Pientka [13], who encode linearity constraints for session types as LF predicates within type judgments rather than using linear contexts. Their predicates track a global resource-usage property of channel variables; ours track a local positional property of a specific occurrence within an axiom. A variable can be eligible in one part of a derivation and ineligible in

another, so the predicates are indexed over derivation terms rather than variables to capture this per-occurrence character.

3.4 Translation Between G_3 and SAX

Both directions of Theorem 1 are recursive functions over derivations using a relation `rel-ctx-S` pairing the two contexts hypothesis by hypothesis. Invertible rules are translated by applying the corresponding constructor directly. The interesting cases are the noninvertible rules, which appear as rules in G_3 but as zero-premise axioms in SAX.

The direction from G_3 to SAX derives each noninvertible rule using cuts and axioms. For instance, $\supset L$ is simulated by two nested cuts against $\supset L^0$:

```
rec seq2sax : (A : [⊢ o]) rel-ctx-S [Γ] [Γ'] → [Γ ⊢ conc A[]]
              → [Γ' ⊢ concS A[]] =
/ total 2 /
fn R D ⇒ case D of
  | [Γ ⊢ ⊃L c1 (\h. c2) H] ⇒
    let [Γ' ⊢ HS]                = hyp2hypS R [Γ ⊢ H] in
    let [Γ' ⊢ c1S]               = seq2sax R [Γ ⊢ c1] in
    let [Γ', x : hypS B[] ⊢ c2S] = seq2sax (cons-S R) [Γ, x : hyp _ ⊢ c2] in
    [Γ' ⊢ cutS (cutS c1S (\h. ⊃L0S h HS[..])) (\h. c2S)]  ...
```

The reverse direction derives each SAX axiom in G_3 using the corresponding rule with identity proofs for the premises. For $\supset L^0$ this is one application of $\supset L$ with two identity sub-derivations:

```
rec sax2seq : (A : [⊢ o]) rel-S-ctx [Γ'] [Γ] → [Γ' ⊢ concS A[]]
              → [Γ ⊢ conc A[]] =
/ total 2 /
fn R D ⇒ case D of
  | [Γ' ⊢ ⊃L0S HS1 HS2] ⇒
    let [Γ ⊢ H1] = hypS2hyp R [Γ' ⊢ HS1] in
    let [Γ ⊢ H2] = hypS2hyp R [Γ' ⊢ HS2] in
    [Γ ⊢ ⊃L (initial H1) (\h. initial h) H2]  ...
```

3.5 Admissibility of Cut and Cut Elimination

Cut admissibility is proved as `cut-admis`, with totality justified by induction on the cut formula. The cases fall into three groups.

First, when the cut variable is in an eligible position in either premise, the result is immediately a snip with no recursive call. For example, when $D = \supset L^0$ the succedent is directly eligible:

```
  %% */_
  | ([Γ ⊢ l⊃L0S h1 h2], [Γ, x : lhypS _ |- E]) ⇒
    [Γ ⊢ lsnip1S (l⊃L0S h1 h2) (elig_c/l⊃L0S h1 h2) (\h. E[..,h])]
```

Second, when the cut formula is principal in both premises, the cut is eliminated by substitution with a recursive call at a strictly smaller type. The $\supset R/\supset L^0$ case requires a three-way split on whether h_1 or h_2 in $\supset L^0$ is the cut variable x, since x may appear in an eligible position in E even when D is $\supset R$:

```
  %% ⊃R/⊃L0
  | ([Γ ⊢ l⊃RS (\h. D1)], [Γ, x : lhypS _ ⊢ l⊃L0S h1 h2]) ⇒
```

```
  (case ([Γ, x : lhypS _ ⊢ h2], [Γ, x : lhypS _ ⊢ h1]) of
    | ([Γ, x:_ ⊢ x],       [Γ, x:_ ⊢ #h1[..]]) ⇒ [Γ ⊢ D1[..,#h1]]
    | ([Γ, x:_ ⊢ #h2[..]], [Γ, x:_ ⊢ x]) ⇒
        [Γ ⊢ lsnip2S (l⊃RS (\h. D1)) (\h. l⊃L0S h #h2[..])
                      (elig_h/l⊃L0S #h2[..])]
    | ([Γ, x:_ ⊢ #h2[..]], [Γ, x:_ ⊢ #h1[..]]) ⇒ [Γ ⊢ l⊃L0S #h1 #h2]
  )
```

Third, when the cut formula is a side formula of the last rule, the cut is commuted past that rule. When E ends with a snip, the outer cut is pushed inside both premises via two recursive calls; if eligibility is lost the snip becomes a general cut, which is permitted since the formula type is then a strict subformula of the cut type. A case worth noting is `_/l ⊗ R0S`, which requires enumerating all combinations of the cut variable x appearing in the argument positions of the tensor axiom, including the degenerate case where both positions are the same variable:

```
| ([Γ ⊢ D'], [Γ, x : lhypS _ ⊢ l⊗R0S h1 h2]) ⇒
  (case ([Γ, x : lhypS _ ⊢ h1], [Γ, x : lhypS _ ⊢ h2]) of
    | ([Γ, x:_ ⊢ #h1[..]], [Γ, x:_ ⊢ #h2[..]]) ⇒ [Γ ⊢ l⊗R0S #h1 #h2]
    | ([Γ, x:_ ⊢ x],       [Γ, x:_ ⊢ #h2[..]]) ⇒
        [Γ ⊢ lsnip2S D' (\h. l⊗R0S h #h2[..])   (elig_h/l⊗R0S/1 #h2)]
    | ([Γ, x:_ ⊢ #h1[..]], [Γ, x:_ ⊢ x]) ⇒
        [Γ ⊢ lsnip2S D' (\h. l⊗R0S #h1[..] h)   (elig_h/l⊗R0S/2 #h1)]
    | ([Γ, x : lhypS A[] ⊢ x], [Γ, x : lhypS A[] ⊢ x]) ⇒
        [Γ ⊢ lsnip2S D' (\h. l⊗R0S h h) elig_h/l⊗R0S/3]
  )
```

Cut elimination over a separate `lconcS-cut` type then applies `cut-admis` at each cut node after recursively processing the sub-derivations:

```
rec cut-elim : {Γ : lctxS} [Γ ⊢ lconcS-cut A[]] → [Γ ⊢ lconcS A[]] =
/ total 1 /
mlam Γ ⇒ fn D ⇒ case D of
  | [Γ ⊢ lcutS-cut D1 (\h. D2)] ⇒
    cut-admis _ _ (cut-elim _ [Γ ⊢ D1]) (cut-elim _ [Γ, x : lhypS _ ⊢ D2])
  | [Γ ⊢ l⊃L0S-cut h1 h2]            ⇒ [Γ ⊢ l⊃L0S h1 h2]
  | [Γ ⊢ l⊗R0S-cut h1 h2]            ⇒ [Γ ⊢ l⊗R0S h1 h2]
  | [Γ ⊢ l⊗LS-cut (\h1. \h2. D1) h3] ⇒
    let [Γ, h1 : lhypS _, h2 : lhypS _ ⊢ D1'] =
      cut-elim _ [Γ, h1 : lhypS _, h2 : lhypS _ ⊢ D1] in
    [Γ ⊢ l⊗LS (\h1. \h2. D1') h3]   ...
```

3.6 Subformula Property

We prove the subformula property in a more general form than the statement in DeYoung et al.: the predicate `subfProp` is parametrized over an arbitrary formula list Σ rather than the fixed set $\Gamma \cup \{A\}$. This generality is necessary because the induction must extend Σ when descending into premises that introduce new hypotheses, and fixing Σ at the top level would require reconstructing the list at each step. The original statement is recovered by instantiating $\Sigma = \Gamma \cup \{A\}$ at the root.

```
inductive subfProp : {Γ : lctxS} {Σ : [⊢ formlist]}
                     {D : [Γ ⊢ lconcS A[]]} ctype =
  | subfProp/initial :
      {#h : #[Γ ⊢ lhypS A[]]} [⊢ inSig A Σ]
      → subfProp [Γ] [⊢ Σ] [Γ ⊢ linitialS #h]
  | subfProp/l⊃L0S :
      {#h1 : #[Γ ⊢ lhypS A[]]} {#h2 : #[Γ ⊢ lhypS (A[] ⊃ B[])]}
      [⊢ inSig A Σ] → [⊢ inSig (A ⊃ B) Σ]
      → subfProp [Γ] [⊢ Σ] [Γ ⊢ l⊃L0S #h1 #h2]
  | subfProp/l⊗R0S :
      {#h1 : #[Γ ⊢ lhypS A[]]} {#h2 : #[Γ ⊢ lhypS B[]]}
      [⊢ inSig A Σ] → [⊢ inSig B Σ]
      → subfProp [Γ] [⊢ Σ] [Γ ⊢ l⊗R0S #h1 #h2]
  | subfProp/lsnip1 :
      (D : [Γ ⊢ lconcS A[]]) (E : [Γ, x : lhypS A[] ⊢ lconcS B[]])
      {e : [Γ ⊢ elig_c D]}
      subfProp [Γ] [⊢ Σ] [Γ ⊢ D] → [⊢ inSig A Σ]
      → subfProp [Γ, x : lhypS A[]] [⊢ Σ] [Γ, x : lhypS A[] ⊢ E]
      → subfProp [Γ] [⊢ Σ] [Γ ⊢ lsnip1S D e (\x. E[..,x])]
  | subfProp/lsnip2 :
      {e : [Γ ⊢ elig_h (\x. E)]}
      subfProp [Γ] [⊢ Σ] [Γ ⊢ D] → [⊢ inSig A Σ]
      → subfProp [Γ, x : lhypS A[]] [⊢ Σ] [Γ, x : lhypS A[] ⊢ E]
      → subfProp [Γ] [⊢ Σ] [Γ ⊢ lsnip2S D (\x. E[..,x]) e]
  ...
```

Three helper lemmas support the main proof. `lookup` extracts a membership proof from the context invariant `ctxInSig [Γ] [⊢ Σ]`, which asserts that every formula type in Γ is a subformula of some element of Σ; given a hypothesis variable in Γ it returns an `inSig` proof for its type. `subf-trans` establishes transitivity of the subformula relation by induction on its second argument. `subf-inSig` promotes a subformula witness into a list membership proof, used wherever the proof descends into a sub-connective: given $B \leq A$ and $A \in \Sigma$ it produces $B \in \Sigma$.

The main proof `subformula-property` proceeds by induction over the structure of the derivation. In the snip cases the `elig` evidence identifies the specific axiom the eligible formula came from, and `subf-inSig` is called with the appropriate subformula witness to produce `inSig A Σ`. The context invariant is then extended via `ctxInSig/cons` and both premises proceed recursively. The case for `elig_h/l⊗R0S/1 #z`, for example, extracts from the `inSig` proof for $A \otimes B$ a membership proof for A:

```
  | [Γ ⊢ lsnip2S D' (\h. E') e] ⇒
    (case [Γ ⊢ e] of
      ...
      | [Γ ⊢ elig_h/l⊗R0S/1 #z] ⇒
        let ih_z   = lookup ci [Γ ⊢ #z] in
        let isig_A = subf-inSig [⊢ subf/⊗1 subf/refl] goal in
        let ci'    = ctxInSig/cons ci isig_A in
        let sp_D   = subformula-property _ [Γ ⊢ D'] ci isig_A in
        let sp_E   = subformula-property _ [Γ, h : lhypS _ ⊢ E'] ci' goal in
```

```
      subfProp/lsnip2 _ sp_D isig_A sp_E
    ...
  )
```

This is precisely the point where the original predicate-on-variables approach would fail: a weakened `elig_h` `h` witness carries no information about which axiom `h` came from, so there would be no basis for the `subf-inSig` call. By indexing over derivation terms instead, the `elig_h/l⊗R0S/1` constructor directly witnesses that the abstracted variable is the first component of a tensor axiom, from which the subformula relationship $A \leq A \otimes B$ is immediate. Extending the mechanization to additional connectives requires adding the corresponding `elig_h` constructors, the matching cases in `cut-admis`, and the appropriate `subf-inSig` calls in `subformula-property`; the structure of each proof obligation is uniform with what is presented here.

4 RELATED WORK

4.1 SAX and Related Systems

Francalaza et. al. [6], present an alternate formulation of SAX embedded in adjoint logic. The resulting computational interpretation models message-passing concurrency: each variable in their system becomes a channel, whose properties are modeled by its adjoint modality, and each consequent becomes a provider of said channels [6]. They present an implementation of this language in Go [6]. Logically then, Pfenning and Pruiksma [12] present SAX with recursion as a unifying framework between message-passing and shared memory. Specifically, they show that the shared-memory semantics and the message-passing semantics have a strong bisimulation on well-typed terms [12].

DeYoung and Pfenning [4] present SNAX, a modification of the standard type theory of SAX that accounts for data layout in shared memory concurrency. It achieves this, in part, by making eligibility a first-class part of the system, rather than a label, as described above; they also add recursion to the system [4]. Ng and Pfenning [10] present SNAX with reuse, a single-threaded language based on SNAX that models memory reuse.

Boyland and Pfenning [1] present an additional adjoint-logical formulation of SAX with bidirectional typing rules. They further show a compilation from adjoint natural deduction [9] to their adjoint system, building the groundwork for a compiler of adjoint-functional programs [1]. Somayyajula and Pfenning [14] applies a type refinement system to SAX, allowing them to reason about the partial correctness of SAX programs. Their framework also allows for the synthesis of SAX with Hoare logic [14].

4.2 Related Mechanizations

Pfenning [11] presents an elegant proof for cut-elimination in the sequent calculus with the same structure as in [5], and our mechanization. He also presents a formalization of this proof in LF [11]. Crary [2] describes a method for encoding linearity in LF without any linear or modal extensions. We used his techniques to in our representation of SAX in Beluga to properly represent eligible formulas in our system. These act similarly to propositions in linear logic as every eligible formula must be used in a SAX proof. We also drew on the techniques described in [13] to properly represent SAX in Beluga.

5 CONCLUSION

We report on a mechanization of the proof theory of Soprillo, a fragment of SAX covering identity, implication, tensor, and snips, in the proof assistant Beluga. Specifically, we mechanize the translation between Soprillo and the sequent calculus G_3, and present mechanized proofs of cut admissibility, cut elimination, and the subformula property.

5.1 Future Work

The most obvious avenue for future work is to extend our mechanization to the complete SAX system, adding connectives such as &, ∨, and **1**. This would further demonstrate the adaptability of our representations and proof techniques. A mechanization of SAX within adjoint logic would similarly allow for meta-theoretic guarantees regarding the message-passing interpretation of SAX.

Another natural step is a mechanization of the shared memory interpretation of SAX, along with its properties including progress and termination. [5] presents a logical relations argument for the latter, which are particularly amenable to mechanization. We suspect that the dynamic semantics of the shared memory interpretation, which takes the form of multiset rewriting rules on configurations (allocated memory cells) can be represented as ordered lists in Beluga. While we intended to explore this path further, our initial challenges at properly representing eligibility in Beluga stalled the project significantly. Following this, a possible extension would be to mechanize SNAX, which builds on SAX by making eligibility a first-class concept in its type theory and shares some of its properties such as type progress and preservation.

REFERENCES

[1] Joanna Boyland and Frank Pfenning. 2025. Compiling adjoint natural deduction to the semi-axiomatic sequent calculus. Abstract, 9th International Workshop on Trends in Linear Logic and its Applications (TLLA 2025), affiliated with FSCD 2025, Birmingham, UK, 19–20 July 2025. https://lipn.univ-paris13.fr/TLLA/2025/abstracts/06-boyland-pfenning.pdf

[2] Karl Crary. 2010. Higher-order representation of substructural logics. In *Proceedings of the 15th ACM SIGPLAN International Conference on Functional Programming* (Baltimore, Maryland, USA) *(ICFP '10)*. Association for Computing Machinery, New York, NY, USA, 131–142. https://doi.org/10.1145/1863543.1863565

[3] H. B. Curry. 1934. Functionality in Combinatory Logic*. *Proceedings of the National Academy of Sciences* 20, 11 (1934), 584–590. https://doi.org/10.1073/pnas.20.11.584 arXiv:https://www.pnas.org/doi/pdf/10.1073/pnas.20.11.584

[4] Henry DeYoung and Frank Pfenning. 2023. Data Layout from a Type-Theoretic Perspective. *Electronic Notes in Theoretical Informatics and Computer Science* 1 (feb 2023). https://doi.org/10.46298/entics.10507 Proceedings of MFPS XXXVIII.

[5] Henry DeYoung, Frank Pfenning, and Klaas Pruiksma. 2020. Semi-Axiomatic Sequent Calculus. In *5th International Conference on Formal Structures for Computation and Deduction (FSCD 2020) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 167)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 29:1–29:22. https://doi.org/10.4230/LIPIcs.FSCD.2020.29

[6] Adrian Francalanza, Gerard Tabone, and Frank Pfenning. 2024. Implementing a Message-Passing Interpretation of the Semi-Axiomatic Sequent Calculus (Sax). In *Coordination Models and Languages - 26th IFIP WG 6.1 International Conference, COORDINATION 2024, Held as Part of the 19th International Federated Conference on Distributed Computing Techniques, DisCoTec 2024, Groningen, The Netherlands, June 17-21, 2024, Proceedings (Lecture Notes in Computer Science, Vol. 14676)*. Springer, 295–313. https://doi.org/10.1007/978-3-031-62697-5_16

[7] Hugo Herbelin. 1995. A Lambda-Calculus Structure Isomorphic to Gentzen-Style Sequent Calculus Structure. In *Computer Science Logic (CSL '94) (Lecture Notes in Computer Science, Vol. 933)*, L. Pacholski and J. Tiuryn (Eds.). Springer, Kazimierz, Poland, 61–75.

[8] W. A. Howard. 1980. The Formulae-as-Types Notion of Construction. In *To H.B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*. Academic Press, 479–490. Original unpublished note from 1969.

[9] Junyoung Jang, Sophia Roshal, Frank Pfenning, and Brigitte Pientka. 2024. Adjoint Natural Deduction (Extended Version). arXiv:2402.01428 [cs.LO] https://arxiv.org/abs/2402.01428 Submitted on 2 Feb 2024.

[10] Daniel Ng. 2024. Memory Reuse in Linear Functional Computation. https://www.cs.cmu.edu/~fp/courses/15417-s25/misc/Ng24.pdf Accessed: 2026-04-21.

[11] Frank Pfenning. 2000. Structural Cut Elimination: I. Intuitionistic and Classical Logic. *Information and Computation* 157, 1–2 (2000), 84–141. https://doi.org/10.1006/inco.1999.2832

[12] Frank Pfenning and Klaas Pruiksma. 2023. Relating Message Passing and Shared Memory, Proof-Theoretically. In *Coordination Models and Languages*, Sung-Shik Jongmans and Antónia Lopes (Eds.). Springer Nature Switzerland, Cham, 3–27.

[13] Chuta Sano, Ryan Kavanagh, and Brigitte Pientka. 2023. Mechanizing Session-Types using a Structural View: Enforcing Linearity without Linearity. *Proc. ACM Program. Lang.* 7, OOPSLA2, Article 235 (Oct. 2023), 26 pages. https://doi.org/10.1145/3622810

[14] Siva Somayyajula and Frank Pfenning. 2023. Dependent Type Refinements for Futures. *Electronic Notes in Theoretical Informatics and Computer Science* Volume 3-Proceedings of... (Nov. 2023). https://doi.org/10.46298/entics.12286

[15] Tesla Zhang, Sonya Simkin, Rui Li, Yue Yao, and Stephanie Balzer. 2025. A Language-Agnostic Logical Relation for Message-Passing Protocols. arXiv:2506.10026 [cs.PL] https://arxiv.org/abs/2506.10026